Bringing Warehouse-Grade Transactions to Open Lakehouses

Multi-Table Transactions on the Iceberg REST Catalog

A live PoC on Apache Polaris

Qiegang Long · Cloudera

Lakehouse Day · Glasgow · 10 Oct 2026

The whole talk in four lines

- **Pain** — strict workloads tear on multi-table reads and partial writes — *solved problems the open lakehouse quietly un-solved.*
- **Gap** — Iceberg's single-table commit *already* spans engines; what's missing is a *multi-table transaction* engines can use — atomic writes **and** one consistent read point.
- **Answer** — a familiar *multi-table, multi-statement* BEGIN / COMMIT coordinated by the catalog — **spanning independent workloads, cross-job or cross-engine.**
- **Proof** — live, across **Spark and Trino**, on one Polaris catalog.

1 · The problem

- A lakehouse rarely has just one table.
- Real datasets are **several tables that only make sense together**.

> The job of multi-table transactions: **keep related tables consistent — both when writing and when reading.**

- Two failure modes show up constantly.

Partial writes — no cross-table atomicity

An *order* and its *order_items* are **one fact split across two tables** — written as two independent commits. Between them, a **window**:

```
order committed, items not yet   → an order with no lines      ✗ not a real order
items committed, order not yet   → lines with no parent order  ✗ orphan rows
```

No commit order is safe — each table needs the other to be true.

> *Writing the bytes was never the hard part. The missing primitive is to publish multiple tables atomically — all visible, or none.*

Torn reads — no cross-table read consistency

Even when *every write is correct*, **reading** several tables can tear:

```
read customers  → 100 customers                          (customer #101 not yet visible)
   …writer commits order #900 AND its new customer #101, atomically…
read orders     → includes order #900 for customer #101

  ⇒ join orders→customers: order #900 points at a customer not in your result
     → a dangling foreign key — the exact 0-vs-1 anomaly the demo proves.  ✗
```

Maddening to debug: **re-running usually “fixes” it** — until it recurs in prod at 3 a.m.

The cost leaks into everything

The platform doesn't guarantee it — so the burden lands on people.

- write-ordering hacks & reconciliation jobs that never fully close the window
- defensive LEFT JOINs and “wait a few minutes” guards in every reader
- flaky, timing-dependent bugs; eroded trust in dashboards
- pipelines forced into one job/engine just to control ordering

> The problem doesn't disappear — it **moves into application code, tests, and on-call**, where it's more expensive and impossible to fully solve.

Why it matters more across engines

- Every distributed query **already reads at one pinned instant** — it just can't share it across engines.
  - **One engine, many workers** — the driver pins; workers read assigned files.
  - **The read gap** — no open catalog lets independent engines share that instant.

- **A shared read point isn't enough** — a snapshot over a torn write is still inconsistent: ETL commits per table, at different instants, so the tables disagree. *(A crash makes it permanent.)*

> **You need both** — a shared read point (**fan-out read**) *and* **fan-out write.** Neither is shareable across **independent jobs — same engine or across engines — today.**

2 Why today's answers fall short

Iceberg's own tools

- **Single-table commit** — atomic per table. But no notion of “these two commits belong together.” Two commits = two events with a visible seam. *Single-table atomicity composed twice is not two-table atomicity.*
- **Multi-table commitTransaction (REST)** — commits several tables **atomically** in one request. But one client must build it all: no BEGIN/COMMIT across statements, no shared read snapshot, no fan-out. *A COMMIT without a BEGIN.*
- **Time travel (AS OF)** — reads one table at a known point. But a consistent multi-table read is two problems: (1) line up each table onto one instant, (2) read it there. AS OF solves (2); for (1) there's no automatic resolver — tags are hand-coordinated, timestamps are the writer's clock.

> Atomic apply exists. A multi-table **transaction** — consistent reads, isolation, fan-out — doesn't.

The three workarounds — each fails differently

Workaround	What it gives up
Eventual consistency (reconcile later)	Correctness — anomaly still observable; you just hope no one looks
Git-like branching (Nessie: commit on branch, merge)	Simplicity — powerful, but a different mental model & ops surface
Single-engine warehouse (one engine's txns)	Openness — transactions only inside that engine

Goal: warehouse-grade semantics without giving up correctness, without a branch/merge workflow, without closing the lakehouse to one engine.

3 The idea

Transactions coordinated by the catalog

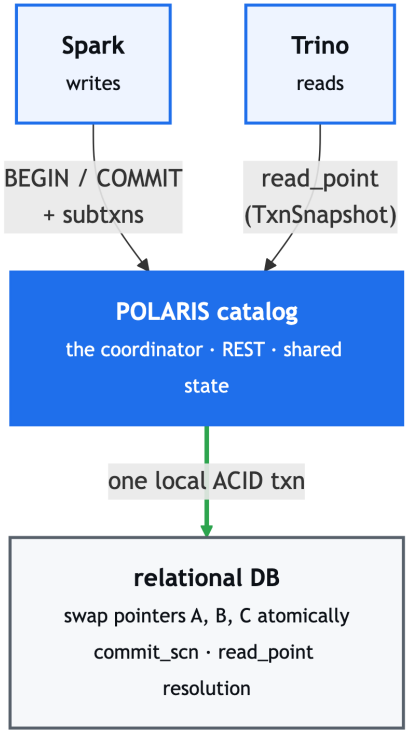
Who is allowed to change tables together?

- A table's “current version” is just a **pointer the catalog publishes**. A commit = the catalog swapping that pointer to a new immutable metadata file.

> **The catalog can flip several table pointers at once — so it's the natural home for a multi-table transaction.**

- The engine sees the catalog as a remote API; it can't referee two writers. The client can't either. **The catalog is already the authority over table pointers — so coordinating there needs no 2PC and no extra coordinator to run.**

The three parts



Any engine with the thin REST client plays — Spark/Trino are just examples. The engine issues BEGIN/COMMIT; the green arrow is the one local ACID txn where atomicity comes from.

4 · How it works — four pillars

1. **One Local Commit** — atomicity from the catalog's DB
2. **Exportable TxnSnapshots** ⭐ — one shareable read handle, any engine
3. **Selectable Isolation Levels** — READ COMMITTED / REPEATABLE READ
4. **Distributable, Retryable Subtransactions** ⭐ — fan per-table units to parallel workers; retry one, not all

⭐ = *the two mechanisms that enable fan-out read and fan-out write*

Pillar 1 · One Local Commit

Claim: stage every table's new metadata, then publish them all in **one local ACID transaction** in the catalog's DB.

```
BEGIN (catalog DB transaction)
   swap pointer for table A → new version
   swap pointer for table B → new version
   record commit position in the commit order
COMMIT      -- commits ⇒ all moved;  rolls back ⇒ none moved
```

Publish multiple tables atomically — as a single COMMIT. Inherit 30+ years of battle-tested ACID durability.

Pillar 2 · Exportable TxnSnapshots ⭐

- **Three words of vocabulary:**
  - **commit_scn** — a commit's position in one ever-increasing sequence (a logical commit clock, not wall time).
  - **read_point** — a cutoff: “everything committed at or before this.”
  - **TxnSnapshot** — the reader's handle: a captured read_point cutoff, no locks, no writes, no transaction. (A read pins a snapshot without opening a transaction — zero server rows.)

Pillar 2 · One read point, resolved by commit order



Green = committed at or before read_point (the candidate set); newest-per-table wins. ③ is ONE transaction publishing customers + products together (one commit_scn). At read_point = 3, orders resolves to ② — ④⑤ land past the cutoff, invisible, no tearing.

Pillar 2 · Why “exportable” is the fan-out read

- The handle is **just a string.** An orchestrator acquires it *once* and hands it to many jobs — this is the fan-out read:

> **Two Spark jobs started minutes apart** — or a Spark job, a Flink job **and** a Trino query — different processes, different machines, all read the **same dataset version** by sharing one read_point.

- That's the read analog of PostgreSQL's pg_export_snapshot() — but across independent processes, up to and including different engines. Exactly what an open lakehouse needs, and what a single-engine warehouse cannot offer to outside engines.

Two “snapshots”: a TxnSnapshot is the txn-level read handle; an Iceberg table snapshot is Iceberg's native per-table point-in-time. The catalog translates the first into the right one of the second, per table.

Pillar 3 · Selectable Isolation Levels

Each reader chooses how strict a view it needs — familiar SQL names. A writer at REPEATABLE READ also gets read-your-own-writes.

Level	What you see	Mechanically
READ COMMITTED	Latest committed data, per statement; reads can differ	a fresh read_point each statement
REPEATABLE READ (default)	One frozen instant for the whole txn, every table	one TxnSnapshot, pinned on first table touch

Pillar 4 · Distributable, Retryable Subtransactions ⭐

Fan-out write — split a transaction into per-table units an orchestrator can **fan out to different workers** (even different engines/machines) that stage **in parallel:**

```
BEGIN TRANSACTION T
 CREATE SUBTRANSACTION FOR orders     → S_orders    → worker 1  ┐ staged in
 CREATE SUBTRANSACTION FOR customers  → S_customers → worker 2  ├ parallel,
 CREATE SUBTRANSACTION FOR products   → S_products  → worker 3  ┘ independently
COMMIT T  ⇒ publish every staged table atomically
```

Because the units are independent, retry stays within the failed subtransaction: if S_customers fails, re-assign just S_customers — possibly to another worker — while orders & products stay staged & untouched. ***Re-run the 3-minute step, not the 3-hour job.***

6 Live Demo

Four demos, one Polaris catalog — Spark and Trino

1 · RC tears, RR doesn't — two Trino readers (READ COMMITTED + REPEATABLE READ); Spark opens a gap between two commits. RC catches the 0→1 dangling FK; RR, pinned before the gap, holds at 0.

2 · No partial write — Spark writes both tables in one atomic transaction; no reader (not even READ COMMITTED) ever catches a gap — counts jump together or not at all.

3 · Fan-out read ⭐ — one exportable snapshot; Spark and Trino pinned to the same instant.

4 · Fan-out write ⭐ — a coordinator hands per-table subtransactions to independent workers.

What the demo proves

- ✓ **RC tears, RR doesn't** — REPEATABLE READ holds one read point for the whole transaction; READ COMMITTED takes a fresh read point per read — so RC saw the writer's gap, RR pinned before it and stayed consistent.
- ✓ **No partial write** — Spark wrote both tables in one atomic transaction; no reader — not even READ COMMITTED reading latest — ever caught a gap. The writer removed the torn state at the source.
- ✓ **Fan-out read** ⭐ — one exportable snapshot handle, passed to Spark and Trino, pinned both engines to the same instant while a writer moved on.
- ✓ **Fan-out write** ⭐ — a coordinator handed per-table subtransactions to independent workers; nothing visible until the parent committed, then everything at once.

7 · How it compares

	Single-engine WH	Nessie (git-like)	This proposal
Cross-table atomicity	✅ (in-engine)	✅ (branch/merge)	✅ (catalog publish)
Consistent multi-table reads	✅ (in-engine)	✅ (pin branch)	✅ (exportable snapshot)
Across independent engines (no 2PC)	❌	✅	✅
Mental model	SQL txns	branch/commit/merge	SQL BEGIN/COMMIT (+savepoints)
Fan to parallel workers + rollback/retry one, not all	n/a	manual	✅ (subtransactions)
No data/format change	n/a	✅ (needs Nessie as catalog)	✅ (any REST catalog; extension)

The differentiator is the combination: warehouse BEGIN/COMMIT (+savepoints), over the open REST catalog, usable by any engine, with distributed-job realism built in.

8 · Let's build it in the open

An early PoC — much of the hard work is still ahead. Where you come in:

> *The community explored this before and it stalled — running code to re-open the problem, together.*

- **Shape the direction** — the design is open; ideas and prior art welcome
- **Review the design & code** — the proposal and PoC are public; a careful read counts
- **DDL inside transactions** — not just DML
- **Snapshot-lifecycle safety** — don't expire pinned snapshots
- **More engines** — beyond Spark + Trino
- **Delegation & hardening** — tokens, conflicts, readiness

A working PoC on Apache Polaris. **Join us.**

Thank you

Questions?

github.com/qlong/lakehouse-transactions

The hub: docs, demos, and pointers to the Polaris & Trino forks

Appendix — Q&A backup

The hard-sounding parts are a small schema + a single indexed lookup.

Appendix · Resolution is one indexed lookup

Engine reads *orders* at **read_point = 105** — the catalog runs exactly:

```
SELECT iceberg_snapshot_id, metadata_location
FROM   committed_table_snapshots
WHERE  realm_id = ? AND catalog_id = ? AND table_uuid = 'orders'
  AND  commit_scn <= 105           -- committed at/before read_point
ORDER BY commit_scn DESC
LIMIT 1;
```

Backed by the **(…, commit_scn DESC)** index → **O(log n)**.

> *Returns a native Iceberg id — the engine reads stock Iceberg with no knowledge of transactions or SCNs. The "resolution algorithm" is a single index-backed lookup.*

Appendix · Locks — opt-in, retry stays safe

Subtransactions give retry; locks decide when a retry can conflict.

- Default **ROW_EXCLUSIVE** — recorded but **non-blocking**, self-compatible (OCC-style). Additive writes stay concurrent — preserving Iceberg's high-throughput ingest.
- **Additive conflict isn't fatal** — catalog merges & retries: re-applies just this write's new data files on top of current state.
- **Destructive op (overwrite/rewrite)?** Client **explicitly opts in** to EXCLUSIVE / ACCESS_EXCLUSIVE — the server never escalates on its own.

- **Lease enforced; reaper ships.** A background reaper (60s) aborts expired transactions and frees their locks — tested end-to-end. Safety never depended on its timing: visibility is the final pointer swap, so a crashed client can at worst hold a lock briefly — **never corrupt a table.** *(One future lever: single-leader gating across nodes.)*

Appendix · The REST protocol delta

Reads standardize now; writes are a vendor extension today.

- **Tier 1 — Iceberg spec, now.** Two *optional* loadTable params, backward-compatible (~20 lines OpenAPI): **txn_snapshot** (opaque string handle) → consistent read; **txn_id** → read-your-own-writes. Trino fans a consistent read out with zero plugin code.
- **Tier 2 — extension now, spec later.** Write verbs under **/polaris/v1/…**. Wire contract: isolation enum on begin; per-table lock enum + 409-retry; subtxn_id (a worker addressing its unit); tokens = capability secrets, not auth.
- **Tier 3 — stays implementation.** Coordinator, schema, lock manager, reaper, commit_scn / read_point — all behind the handles, never on the wire.

Appendix · Is it “chatty”? — Reads

Turning transactions on adds zero extra catalog calls to reads. Baseline = plain Iceberg REST catalog; one round-trip = one client→catalog call.

Read scenario	Plain REST catalog	Polaris + transactions
READ COMMITTED · read N tables	N	N — identical
REPEATABLE READ · read N tables	can't offer consistency	N + 1

READ COMMITTED read = plain catalog, exactly — we open no transaction. REPEATABLE READ adds one lazy snapshot-pin per session (a stateless, server-minted handle) — not per statement, no per-read DB row.

Appendix · Is it “chatty”? — Writes

A plain REST catalog can already commit two tables atomically — a bundled commitTransaction, ~3 calls. A transaction costs one more call and adds what the bundle can't: consistent reads across tables, isolation levels, and fan-out to independent workers. Staging goes straight to object storage — not a catalog call, either side.

Write scenario	Plain REST catalog	Polaris + transactions
Two tables, separate commits	4 — not atomic	—
Two tables, atomic	3 — bundled commitTransaction: atomic, no shared read point	4 — BEGIN/COMMIT: atomic + consistent reads + isolation
Fan-out write, S workers	can't express — one client builds it all	total = 2 + 3·S calls critical path ≈ 5 (constant in S)

Plain = 2 loadTable + 1 bundled commit; ours = begin + 2 pinned resolves + 1 commit — the extra call buys one consistent read point and an isolation level. Fan-out: subtransactions run in parallel → critical path ≈ 5.